

0.87.4. Agrandir une lettre : `texteGrand` — vingt fois

La plus grande taille : `texteGrand(0, 0, "A", 20)`. Le A remplit tout l'écran, 160 × 140 pixels.

1. Ce qu'il faut comprendre

La taille la plus grande, 20 : chaque pixel du A devient un carré de 20 × 20 pixels. La lettre fait **160 pixels de large** (toute la largeur de l'écran) et **140 de haut** : elle remplit l'écran.

Pourquoi 20 au plus : une lettre de la police fait **7 pixels de haut**, et l'écran **144**. $7 \times 20 = 140$: elle tient encore entière. À 21, $7 \times 21 = 147$: le bas du A sortirait de l'écran. Le compilateur refuse donc au-delà de 20, et dit pourquoi.

En largeur aussi, tout juste : une case de lettre fait 8 pixels ; $8 \times 20 = 160$, la largeur exacte de l'écran. Pour tenir, le A géant commence en (0, 0).

Ça ne coûte presque rien en dessins : à cette taille, presque toutes les tuiles sont toutes pleines ou toutes vides. Le A géant n'en demande que 9 différentes.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrand>` → [0.86.3. La fonction `texteGrand\(\)` — un texte agrandi](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2392 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          55 o  ← la console, à l'allumage — la mise en route de la console
texte_grand_0()    979 o  ← main() — ta fonction
main()             16 o  ← démarrage — ta fonction
VBlank             8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o  ← image() — attend l'image suivante
AttendreVBlank     16 o  ← démarrage — attend que l'écran soit libre
AdresseCase       10 o  ← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles       18 o  ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o  ← démarrage — vide les deux cartes de l'écran
Tuiles            848 o  ← CopierTuiles — la police entière, et 9 tuiles dessinées
GRAND_TABLE_0_0_0  20 o  ← poser() — une table gravée
GRAND_TABLE_0_1_1  20 o  ← poser() — une table gravée
GRAND_TABLE_0_2_2  20 o  ← poser() — une table gravée
GRAND_TABLE_0_3_3  20 o  ← poser() — une table gravée
GRAND_TABLE_0_4_4  20 o  ← poser() — une table gravée
GRAND_TABLE_0_5_5  20 o  ← poser() — une table gravée
GRAND_TABLE_0_6_6  20 o  ← poser() — une table gravée
GRAND_TABLE_0_7_7  20 o  ← poser() — une table gravée
GRAND_TABLE_0_8_8  20 o  ← poser() — une table gravée
GRAND_TABLE_0_9_9  20 o  ← poser() — une table gravée
```

```

GRAND_TABLE_0_10_10  20 o ← poser() – une table gravée
GRAND_TABLE_0_11_11  20 o ← poser() – une table gravée
GRAND_TABLE_0_12_12  20 o ← poser() – une table gravée
GRAND_TABLE_0_13_13  20 o ← poser() – une table gravée
GRAND_TABLE_0_14_14  20 o ← poser() – une table gravée
GRAND_TABLE_0_15_15  20 o ← poser() – une table gravée
GRAND_TABLE_0_16_16  20 o ← poser() – une table gravée
GRAND_TABLE_0_17_17  20 o ← poser() – une table gravée
GRAND_TABLE_0_18_18  20 o ← poser() – une table gravée
GRAND_TABLE_0_19_19  20 o ← poser() – une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
    poser()          × 20    618 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (20 o),
GRAND_TABLE_0_1_1 (20 o), GRAND_TABLE_0_2_2 (20 o), GRAND_TABLE_0_3_3 (20 o), GRAND_TABLE_0_4_4 (20 o),
GRAND_TABLE_0_5_5 (20 o), GRAND_TABLE_0_6_6 (20 o), GRAND_TABLE_0_7_7 (20 o), GRAND_TABLE_0_8_8 (20 o),
GRAND_TABLE_0_9_9 (20 o), GRAND_TABLE_0_10_10 (20 o), GRAND_TABLE_0_11_11 (20 o), GRAND_TABLE_0_12_12 (20 o),
GRAND_TABLE_0_13_13 (20 o), GRAND_TABLE_0_14_14 (20 o), GRAND_TABLE_0_15_15 (20 o), GRAND_TABLE_0_16_16 (20 o),
GRAND_TABLE_0_17_17 (20 o), GRAND_TABLE_0_18_18 (20 o), GRAND_TABLE_0_19_19 (20 o)
    image()          × 1     3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
    texte_grand_0() × 1     3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texteGrand> // écrit un texte agrandi, de 1 à 20 fois
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

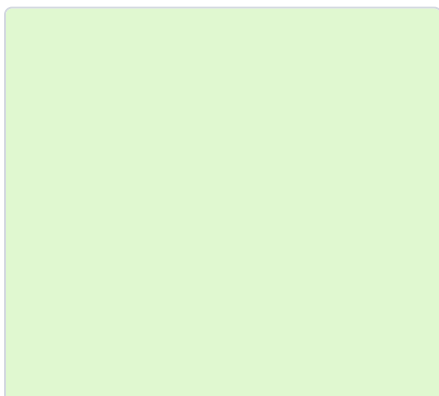
int main() {
    // La plus grande taille : 20. Le A fait 20 × 20 cases, tout l'écran.
    // 8 pixels × 20 = 160 : toute la largeur
    // 7 pixels × 20 = 140 : presque toute la hauteur (144)
    texteGrand(0, 0, "A", 20);

    while (true) {
        image();
    }
}

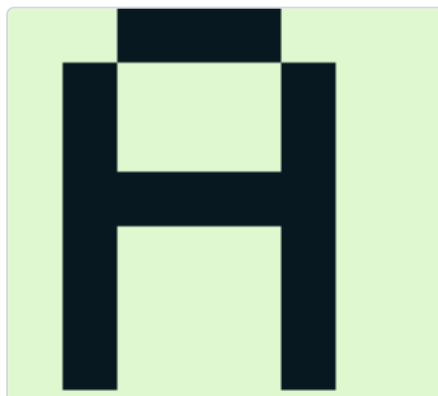
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A immense, qui remplit tout l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A géant occupe une grande part de l'écran (140 cases pleines)

.....

Ensuite : **0.87.5. La fonction `texteGrandS()` — un grand texte qui va à la ligne** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++